TT-XLA Installation

- Purpose
- Docker Setup
  - 1. Pull Docker Image
  - 2. Define Environment Variables
  - 3. Launch Docker Container
  - 4. Configure Environment Variables in Docker
- Build TT-XLA
  - 1. Clone Required Repositories
    - For BOS A0
    - For Tenstorrent
  - 2. Build TT-MLIR Toolchain
  - 3. Build TT-XLA
    - For BOS MLIR + A0
    - For Tenstorrent MLIR
    - Note
- Test
- Troubleshooting
  - Github Acess
  - Python Version
  - Mismatch SFPI Toolchain
  - Build Tools TT-MLIR
- References

Purpose

This guide explains how to set up a **TT-XLA development environment** on BOS A0 or Tenstorrent device using Docker.

Docker Setup

1. Pull Docker Image

If the Docker image is **not already present**, pull it first:

```
docker pull ghcr.io/tenstorrent/tt-xla/tt-xla-ci-ubuntu-22-04:latest
```

2. Define Environment Variables

Before launching the container, define the following variables on the host:

```
DOCKER_NAME="<docker_name>"
PORT_NUMBER=<port_number>

```

```
HOME_WORKDIR="<host_workspace_path>"
DATA_DIR="<host_data_path>"
CONTAINER_WORKDIR="/workspace"

IMAGE_NAME="ghcr.io/tenstorrent/tt-xla/tt-xla-ci-ubuntu-22-04:latest"

SHM_SIZE="128g"

# if using TT device
DEVICE_PATH="/dev/tenstorrent/<device_id>"
# if using A0 device
DEVICE_PATH="/dev/bos/<device_id>"
```

3. Launch Docker Container

Run the following command to start the container:

```
docker run -itd \
    --name $DOCKER_NAME \
    -p $PORT_NUMBER:$PORT_NUMBER \
    -v $HOME_WORKDIR:$CONTAINER_WORKDIR \
    -v $DATA_DIR:/data \
    -v /dev/hugepages:/dev/hugepages \
    -v /dev/hugepages-1G:/dev/hugepages-1G \
    --device $DEVICE_PATH:$DEVICE_PATH \
    --shm-size $SHM_SIZE \
    --cap-add ALL \
    --ipc=host \
    --restart unless-stopped \
    $IMAGE_NAME bash
```

4. Configure Environment Variables in Docker

Inside the container, update `~/.bashrc`:

```
export TT_XLA_RUNTIME_ROOT=/workspace/xla-dev/tt-xla
export PROJECT_SOURCE_DIR=/workspace/xla-dev/tt-xla
export TT_MLIR_RUNTIME_ROOT=$TT_XLA_RUNTIME_ROOT/third_party/tt-mlir/src/tt-mlir
export TT_METAL_RUNTIME_ROOT=$TT_MLIR_RUNTIME_ROOT/third_party/tt-metal/src/tt-metal
export TTMLIR_TOOLCHAIN_DIR=/opt/ttmlir-toolchain
```

Build TT-XLA

1. Clone Required Repositories

Create a workspace directory and clone the necessary repositories:

For BOS A0

```
# Create xla folder
mkdir -p /workspace/xla-dev && cd /workspace/xla-dev

# Clone necessary repository
git clone --branch develop git@github.com:bos-semi/tt-mlir.git # for toolchain
git clone --branch release/a0 git@github.com:bos-semi/tt-xla.git
```

For Tenstorrent

```
# Create xla folder
mkdir -p /workspace/xla-dev && cd /workspace/xla-dev

# Clone necessary repository
git clone https://github.com/tenstorrent/tt-mlir.git  # for toolchain
git clone https://github.com/tenstorrent/tt-xla.git
```

2. Build TT-MLIR Toolchain

```
cd /workspace/xla-dev/tt-mlir

export TTMLIR_TOOLCHAIN_DIR=/opt/ttmlir-toolchain

mkdir -p $TTMLIR_TOOLCHAIN_DIR
# chown -R "${USER}" "${TTMLIR_TOOLCHAIN_DIR}"

cmake -B env/build env \
  -DCMAKE_C_COMPILER=clang \
  -DCMAKE_CXX_COMPILER=clang++

# If build fail, please look Troubleshooting section
cmake --build env/build
```

Activate the toolchain environment:

```
source env/activate
```

Ensure Python version is **3.12**:

```
python --version
```

3. Build TT-XLA

```
cd $TT_XLA_RUNTIME_ROOT
git submodule update --init --recursive
source venv/activate
```

Then build:

For BOS MLIR + A0

```
# Configure the build for A0
cmake -G Ninja -B build \
-DTT_MLIR_VERSION=develop \
-DUSE_BOS_SEMI_TTMLIR=ON \
-DUSE_CUSTOM_TT_MLIR_VERSION=ON \
-DUSE_BOS_REPO=ON

# Build the project
cmake --build build

# If you faced error, see Troubleshooting section
```

XLA offers many options to build TT-MLIR tools

```
cmake -G Ninja -B build \
-DTT_MLIR_VERSION=develop \
-DUSE_BOS_SEMI_TTMLIR=ON \
```

```
-DUSE_CUSTOM_TT_MLIR_VERSION=ON \
-DUSE_BOS_REPO=ON \
-DTTMLIR_ENABLE_BINDINGS_PYTHON=ON \
-DTTMLIR_ENABLE_PERF_TRACE=ON \
-DTTXLA_ENABLE_TOOLS=ON
```

You should alias to use convenniently

```
alias ttmlir-opt='$TT_MLIR_RUNTIME_ROOT/build/bin/ttmlir-opt'
alias ttmlir-translate='$TT_MLIR_RUNTIME_ROOT/build/bin/ttmlir-translate'
```

For Tenstorrent MLIR

```
cmake -G Ninja -B build
# Build the project
cmake --build build

# If you faced error, see Troubleshooting section
```

After successfully build, add this line to `~/.bashrc`

```
cd $TT_XLA_RUNTIME_ROOT
source $TT_XLA_RUNTIME_ROOT/venv/activate
```

For a debug build:

```
cmake -G Ninja -B build -DCMAKE_BUILD_TYPE=Debug
```

Note

When using `tt-metal-e2` and running build, you will encounter an issue:

```
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/fmt/69912fb6b71fcb1f7e5deca191a2bb4748c4e7b6/include/fmt/base.h:2235:45: error
: implicit instantiation of undefined template 'fmt::detail::type_is_unformattable_for<std::vector<unsigned int>, char>'
 2235 |     type_is_unformattable_for<T, char_type> _;
      |                                             ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/fmt/69912fb6b71fcb1f7e5deca191a2bb4748c4e7b6/include/fmt/base.h:2208:44: note:
 in instantiation of function template specialization 'fmt::detail::value<fmt::context>::value<std::vector<unsigned int>, 0>' requested here
 2208 |   FMT_CONSTEXPR20 FMT_INLINE value(T& x) : value(x, custom_tag()) {}
      |                                             ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/fmt/69912fb6b71fcb1f7e5deca191a2bb4748c4e7b6/include/fmt/base.h:2781:12: note:
 in instantiation of function template specialization 'fmt::detail::value<fmt::context>::value<std::vector<unsigned int>, 0>' requested here
 2781 |   return {{args...}};
      |          ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/spdlog/b1c2586bb5c35a7929362e87f62433eb68206873/include/spdlog/logger.h:328:5
: note: in instantiation of function template specialization 'fmt::make_format_args<fmt::context, std::vector<unsigned int>, 1, 0, 15ULL>' requested here
  328 |             fmt::vformat_to(fmt::appender(buf), fmt, fmt::make_format_args(args...));
      |                                                      ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/spdlog/b1c2586bb5c35a7929362e87f62433eb68206873/include/spdlog/logger.h:80:9:
note: in instantiation of function template specialization 'spdlog::logger::log_<std::vector<unsigned int> &>' requested here
   80 |         log_(loc, lvl, details::to_string_view(fmt), std::forward<Args>(args)...);
      |         ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/tt_metal/common/core_assignment.cpp:202:5: note: in instantiation of function template
pecialization 'spdlog::logger::log<std::vector<unsigned int> &>' requested here
  202 |     log_info(tt::LogMetal, "Dram Interface Workers: {}", worker_phy_x);
      |     ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/tt-logger/d2339ce68562cae34cd95f3fece7fd94eb0529b7/include/tt-logger/tt-logger
.hpp:315:33: note: expanded from macro 'log_info'
  315 | #    define log_info(type, ...) SPDLOG_LOGGER_INFO(tt::LoggerRegistry::instance().get(type), __VA_ARGS__)
      |                                ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/spdlog/b1c2586bb5c35a7929362e87f62433eb68206873/include/spdlog/spdlog.h:314:9
 note: expanded from macro 'SPDLOG_LOGGER_INFO'
  314 |         SPDLOG_LOGGER_CALL(logger, spdlog::level::info, __VA_ARGS__)
      |         ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/spdlog/b1c2586bb5c35a7929362e87f62433eb68206873/include/spdlog/spdlog.h:288:19
: note: expanded from macro 'SPDLOG_LOGGER_CALL'
  288 |         (logger)->log(spdlog::source_loc{__FILE__, __LINE__, SPDLOG_FUNCTION}, level, __VA_ARGS__)
      |                  ^
/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/third_party/tt-metal/src/tt-metal/.cpmcache/fmt/69912fb6b71fcb1f7e5deca191a2bb4748c4e7b6/include/fmt/base.h:2075:45: note:
 template is declared here
```

buid_xla_tt_met...ror.txt
26 Mar 2026, 12:08 AM

diff_tt_metal_build_xla
26 Mar 2026, 12:08 AM

You must manually modify `$TT_METAL_RUNTIME_ROOT/tt_metal/common/core_assignment.cpp`

Within the function `get_optimal_dram_to_physical_worker_assignment(...)`, apply the following changes:

- **Mark unused parameters with `[[maybe_unused]]` (lines 159–165):**

```
[[maybe_unused]] const std::vector<CoreCoord>& dram_phy_coords,
[[maybe_unused]] uint32_t full_grid_size_x,
[[maybe_unused]] uint32_t full_grid_size_y,
[[maybe_unused]] std::vector<uint32_t> worker_phy_x,
[[maybe_unused]] std::vector<uint32_t> worker_phy_y
```

- **Comment out debug logs (lines 198–199):**

```
// log_info(tt::LogMetal, "Dram Interface Workers: {}", full_grid_size_x);
// log_info(tt::LogMetal, "Dram Interface Workers: {}", worker_phy_x);
```

and

```
git add tt_metal/common/core_assignment.cpp
git commit -m "resolve"
```

come back to `TT_XLA_RUNTIME_ROOT` and rebuild, it will work fine

Test

```
pytest $TT_XLA_RUNTIME_ROOT/tests/benchmark/test_vision.py::test_resnet50 -sv
```

Troubleshooting

Github Acess

Ensure that your machine can authenticate with GitHub via SSH. This is required to access the BOS repositories.

```
ssh -T git@github.com

# If SSH is configured correctly, you should see a message similar to:
# Hi <username>! You've successfully authenticated, but GitHub does not provide shell access.
```

Create an SSH Key (if you don't have one)

```
# If the SSH check fails, generate a new SSH key:

```

```
ssh-keygen -t ed25519 -C "your_email@example.com"

# When prompted for the file location, press Enter to accept the default:

# ~/.ssh/id_ed25519       # private key
# ~/.ssh/id_ed25519.pub   # public key

# Display the public key:

cat ~/.ssh/id_ed25519.pub

# Copy the output and add it to your GitHub account:
# GitHub → Settings → SSH and GPG keys → New SSH key

# Start the SSH agent and add your key:

eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Then verify the connection again:

ssh -T git@github.com
```

Python Version

Might be need install 3.12 || 3.11

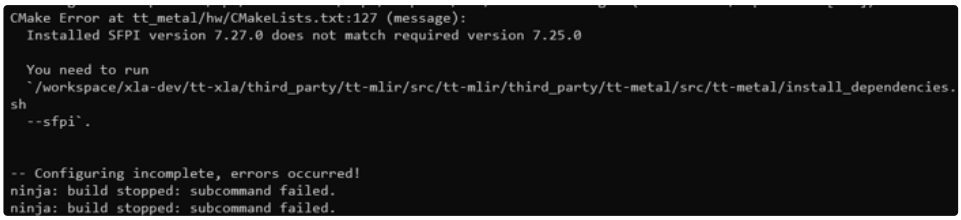
If `python3.11` is not available in the environment, create a symbolic link to an existing Python installation (e.g., Python 3.12):

```
sudo apt-get update
sudo apt-get install -y python3.12 python3.12-venv
ln -s /usr/bin/python3.12 /usr/bin/python3.11
```

Then rebuild the project:

```
cmake --build env/build
```

Mismatch SFPI Toolchain




```
$TT_METAL_RUNTIME_ROOT/install_dependencies.sh --sfpi
```

Build Tools TT-MLIR

After building tools for TT-MLIR, if you encountered this issue

```
((venv) ) root@ebb86305068b:/workspace/xla-dev/tt-xla# ttrt
Warning: not importing testing submodule since project was not built with runtime testing enabled. To enable, rebuil
d with: -DTTMLIR_ENABLE_RUNTIME_TESTS=ON
Traceback (most recent call last):
  File "/workspace/xla-dev/tt-xla/venv/bin/ttrt", line 3, in <module>
    from ttrt import main
  File "/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/build/python_packages/ttrt/__init__.py", line 18,
in <module>
    from ttrt.common.api import API
  File "/workspace/xla-dev/tt-xla/third_party/tt-mlir/src/tt-mlir/build/python_packages/ttrt/common/api.py", line 15
, in <module>
    from pkg_resources import get_distribution
ModuleNotFoundError: No module named 'pkg_resources'
```

Run

```
python -m pip install --force-reinstall "setuptools<82"
```

References

I followed these documents to install TT-XLA:

- [Getting Started with Building from Source - tt-xla documentation]
- [TT-Forge Tools Installation - AI/Multimedia - Confluence]